Rapport de Projet

Apprentissage par Renforcement appliqué à Flappy Bird

Deux agents distincts : Monte Carlo Control & SARSA(λ) True Online

Léo Pommier

2025–2026

Table des matières

1 Introduction **2**
1.1 Environnement : `TextFlappyBirdEnvSimple` 2

2 Agent 1 — Monte Carlo Control (On-Policy) **2**
2.1 Principe de l'algorithme 2
2.2 Hyperparamètres 3
2.3 Résultats d'entraînement 3
2.4 Politique et fonction de valeur 4
2.5 Analyse 5

3 Agent 2 — True Online SARSA(λ) **5**
3.1 Principe de l'algorithme 5
3.2 Approximation linéaire de la fonction de valeur 5
3.3 Hyperparamètres 6
3.4 Résultats d'entraînement 6
3.5 Politique et fonction de valeur 7
3.6 Analyse 8

4 Comparaison des deux agents **8**
4.1 Tableau récapitulatif 8
4.2 Comparaison des politiques et des histogrammes 9
4.3 Compromis biais–variance 9
4.4 Généralisation à de nouvelles configurations 10

5 Structure du code **10**
5.1 Interface commune 10

6 Conclusion et réponses à certaines questions **11**

1. Introduction

Ce projet s'inscrit dans le cadre du cours d'Apprentissage par Renforcement (RL). L'objectif est de concevoir et comparer deux agents capables de jouer au jeu Flappy Bird dans un environnement textuel simplifié, en appliquant deux familles d'algorithmes RL distinctes.

Flappy Bird constitue un environnement de test pertinent pour le RL : l'espace d'états est continu par nature (distances à l'obstacle), les conséquences d'une action sont immédiates (collision ou survie), et la politique optimale est non triviale mais appréhensible.

1.1. Environnement : `TextFlappyBirdEnvSimple`

L'environnement est implémenté comme un environnement Gymnasium (OpenAI Gym). À chaque pas de temps, l'agent reçoit une observation sous forme de tuple (`dx`, `dy`) :

— **dx** : distance horizontale entre l'oiseau et le prochain tuyau.

— **dy** : distance verticale entre l'oiseau et le centre du passage entre les tuyaux, (**dy** est négatif lorsque l'oiseau se trouve au dessus du centre du passage).

L'espace d'actions est discret et binaire :

— **Action 0 — Idle** : l'oiseau tombe sous l'effet de la gravité.

— **Action 1 — Flap** : l'oiseau effectue un battement d'ailes (monte).

La récompense est +1 à chaque pas de temps où l'oiseau reste en vie. L'épisode se termine lorsque l'oiseau heurte un tuyau, le sol ou le plafond.

J'ai ajouté une limitation de 10 000 steps au jeu pour controler la durée des parties jouées par nos agents.

Table 1 – Paramètres de configuration de l'environnement.

`Paramètre`	Valeur par défaut	Description
`height`	15	Hauteur de la grille (en cases)
`width`	20	Largeur de la grille (en cases)
`pipe_gap`	4	Taille de l'ouverture entre les tuyaux

2. Agent 1 — Monte Carlo Control (On-Policy)

2.1. Principe de l'algorithme

Le Monte Carlo Control est une méthode sans modèle (*model-free*) qui apprend directement à partir d'épisodes complets. Les mises à jour ne s'effectuent qu'en fin d'épisode, une fois le retour cumulé G connu pour chaque paire (état, action).

L'algorithme implémenté est le **First-Visit MC Control epsilon-soft** :

1. Générer un épisode complet en suivant la politique ε-greedy courante.

2. Calculer le retour Monte Carlo :

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k \, R_{t+k+1}$$

3. Pour chaque *première visite* d'une paire (s, a), mettre à jour $Q(s, a)$:

$$Q(s,a) \leftarrow Q(s,a) + \frac{1}{N(s,a)} \big[G - Q(s,a) \big]$$

4. Dériver la politique greedy à partir de Q.

2.2. Hyperparamètres

Table 2 – Hyperparamètres de l'Agent 1 (MC Control).

Hyperparamètre	Valeur	Rôle
`episodes`	5 000	Nombre d'épisodes d'entraînement
`gamma`	0.99	Facteur de réduction
`epsilon`	$0.20 \rightarrow 0.05$	Taux d'exploration
`epsilon_decay`	0.999	Décroissance de ε par épisode
`eval_episodes`	20	Épisodes pour l'évaluation greedy

2.3. Résultats d'entraînement

La figure 1 présente l'évolution du retour moyen greedy. On observe une progression régulière jusqu'à l'épisode 2 000, après lequel la politique converge et atteint le plafond de 10 000.

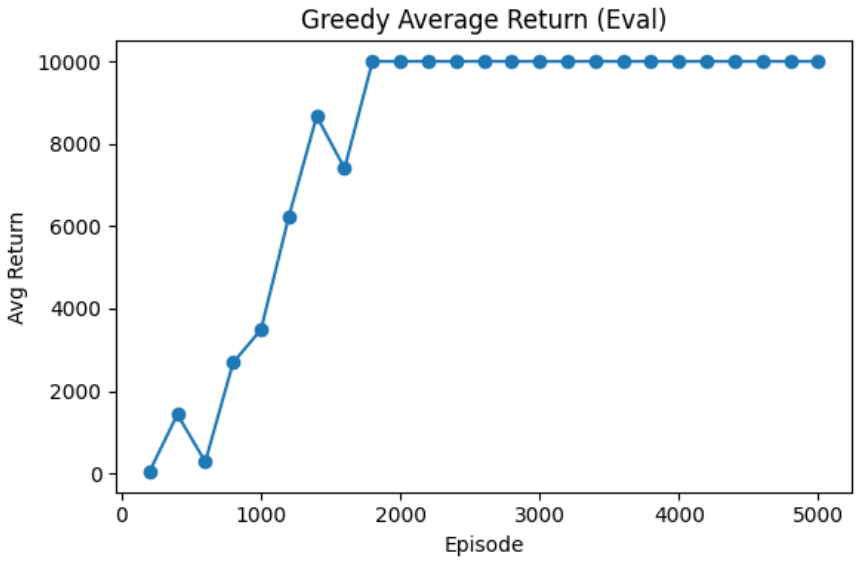
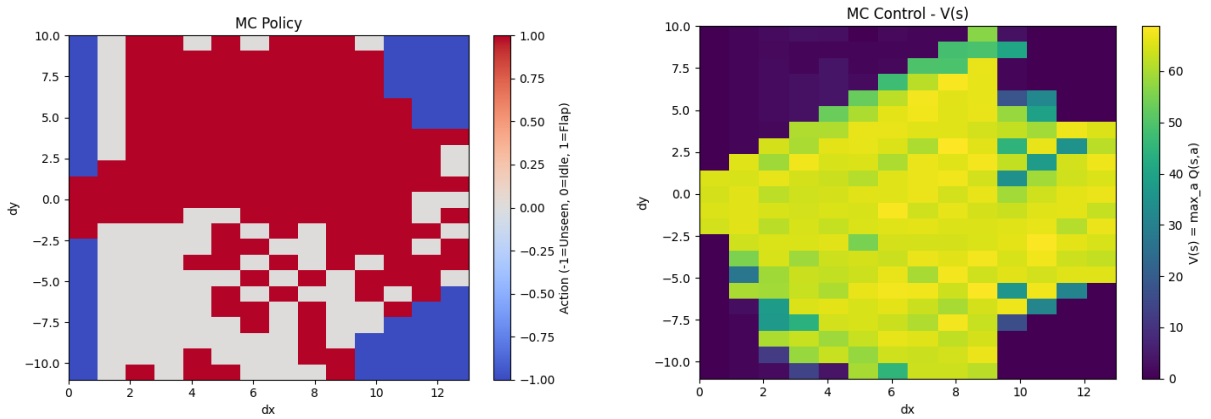


Figure 1 – Courbe d'apprentissage de l'agent MC Control.

2.4. Politique et fonction de valeur



(a) Politique greedy (-1 = Unseen 0 = Idle, 1 = Flap).

(b) Fonction de valeur $V(s) = \max_a Q(s, a)$.

FIGURE 2 – Politique et fonction de valeur apprises par l'agent MC Control.

La politique MC (figure 2a) présente un aspect **fragmenté** : des îlots de décisions *Flap* et *Idle* coexistent dans toutes les régions de l'espace, y compris celles où l'on attendrait une décision homogène. Ceci est caractéristique d'une représentation tabulaire sur un espace partiellement couvert. Les zones bleues permettent de se rendre compte des zones inateignables pour l'oiseau.

La fonction de valeur (figure 2b) est en revanche cohérente : les valeurs les plus élevées se concentrent dans la bande centrale ($dy \in [-5, 5]$), correspondant aux états les plus fréquemment visités par une politique performante.

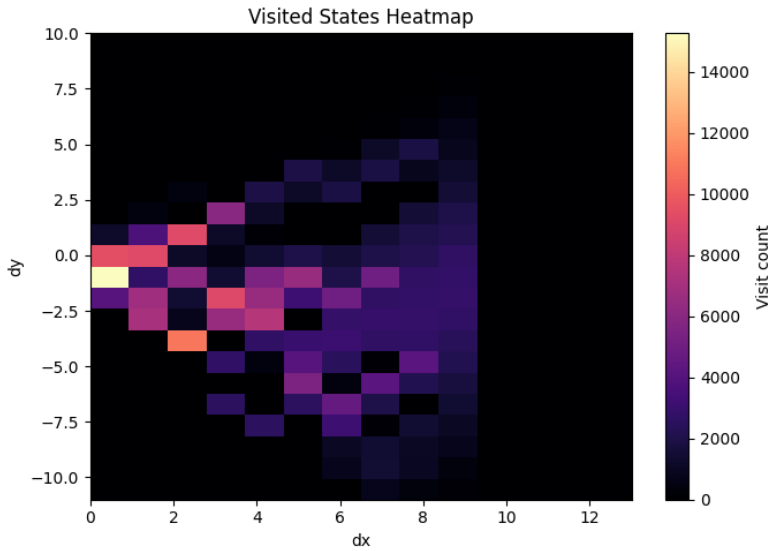


FIGURE 3 – Histogramme des états visités de l'agent MC Control en évaluation greedy (200 épisodes).

Cet histogramme confirme que les régions centrales sont le plus visités.

2.5. Analyse

Avantages

— Convergence garantie vers Q^* sous couverture suffisante de l'espace d'états.

— Mises à jour **sans biais** : les retours sont calculés sur trajectoires réelles.

— Robustesse : une fois convergée, la politique est très stable.

Limitations

— Nécessité d'attendre la fin de chaque épisode — apprentissage plus lent.

— **Variance élevée** des estimations pour les épisodes longs.

— Politique fragmentée dans les zones peu visitées.

3. Agent 2 — True Online SARSA(λ)

3.1. Principe de l'algorithme

SARSA(λ) est un algorithme de différence temporelle (TD) qui combine les mises à jour **en ligne** avec des **traces d'éligibilité**. L'implémentation utilise la variante **True Online SARSA(λ)** :

$$\mathbf{z} \leftarrow \gamma\lambda\,\mathbf{z} + \left(1 - \alpha\,\gamma\lambda\,\mathbf{z}^\top\mathbf{x}\right)\mathbf{x} \tag{1}$$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha\,(\delta + Q - Q_{\text{old}})\,\mathbf{z} - \alpha\,(Q - Q_{\text{old}})\,\mathbf{x} \tag{2}$$

où $\delta = R + \gamma Q' - Q$ est l'erreur TD.

3.2. Approximation linéaire de la fonction de valeur

La fonction Q est approximée par un modèle linéaire $q(s, a; \mathbf{w}) = \mathbf{w}^\top\mathbf{x}(s, a)$, avec les features polynomiaux :

$$\boldsymbol{\phi}(s) = \left[1,\ \tilde{dx},\ \tilde{dy},\ \tilde{dx}\cdot\tilde{dy},\ \tilde{dx}^2,\ \tilde{dy}^2\right]^\top \in \mathbb{R}^6$$

Les features d'action sont obtenus par concaténation sélective, donnant $\mathbf{x}(s, a) \in \mathbb{R}^{12}$ (**12 paramètres au total**).

3.3. Hyperparamètres

TABLE 3 – Hyperparamètres de l'Agent 2 (True Online SARSA(λ)).

Hyperparamètre	**Valeur**	**Rôle**
`alpha`	0.1	Taux d'apprentissage
`gamma`	0.99	Facteur de réduction
`lambda_`	0.9	Paramètre de trace d'éligibilité
`epsilon`	$0.20 \rightarrow 0.05$	Taux d'exploration
`keep_best`	`True`	Sauvegarde du meilleur snapshot

3.4. Résultats d'entraînement

La courbe d'apprentissage (figure 4) révèle un comportement très différent de celui de MC Control : les performances oscillent fortement tout au long de l'entraînement, avec des pics atteignant le score maximal (épisodes 2 200 et 3 400) mais sans stabilisation durable.

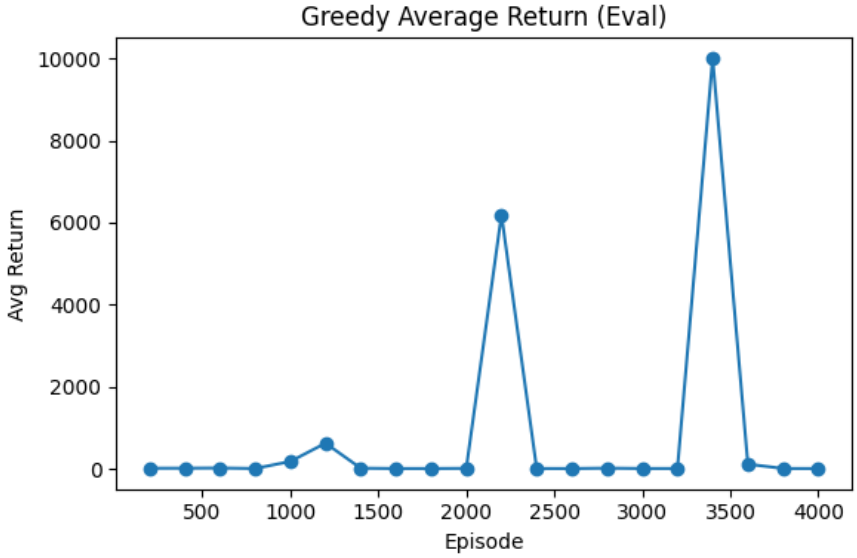
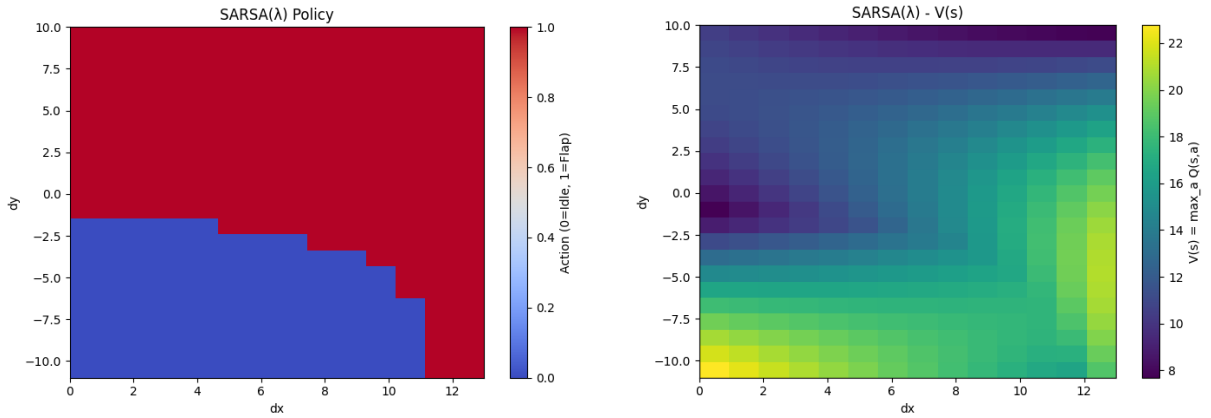


FIGURE 4 – Courbe d'apprentissage de l'agent SARSA(λ).

Cette instabilité est caractéristique des méthodes à approximation de fonction : les mises à jour continues de $\mathbf{w}$ peuvent dégrader une bonne politique dans des régions importantes de l'espace d'états. Le mécanisme`keep_best` permet de conserver le meilleur snapshot observé.Nous avons essayé de diminuer alpha à 0.001 et lambda à 0.7 sans obtenir d'amélioration sur la stabilité du learning. Une technique pour améliorer le learning pourrait être de clipper les changements de poids. Etant donné que notre training trouve déja une policy optimale nous n'avons pas plus poussé l'amélioration du training.

3.5. Politique et fonction de valeur



(a) Politique greedy (0 = Idle, 1 = Flap).

(b) Fonction de valeur $V(s)$.

FIGURE 5 – Politique et fonction de valeur apprises par l'agent SARSA(λ).

Contrairement à la politique MC, la politique SARSA(λ) (figure 5a) présente une **frontière de décision lisse et monotone** : conséquence directe de l'approximation linéaire, qui impose une structure globale à la politique.

La fonction de valeur (figure 5b) est également lisse, avec un gradient clair : les valeurs augmentent à mesure que dx croît (plus loin du tuyau implique plus de temps pour survivre). Cette Q-value est plus dur à expliquer que celle de l'agent MC, il est dur de comprendre pourquoi le coin inférieur gauche a une forte Q-value, une hypothèse probable est que la q-value étant une fonction linéaire les extrémités sont plus facilement récompsées. On constate toute fois en examinant la policy, que le coins inférieur gauche n'est pas un état que la policy va pousser à visiter.

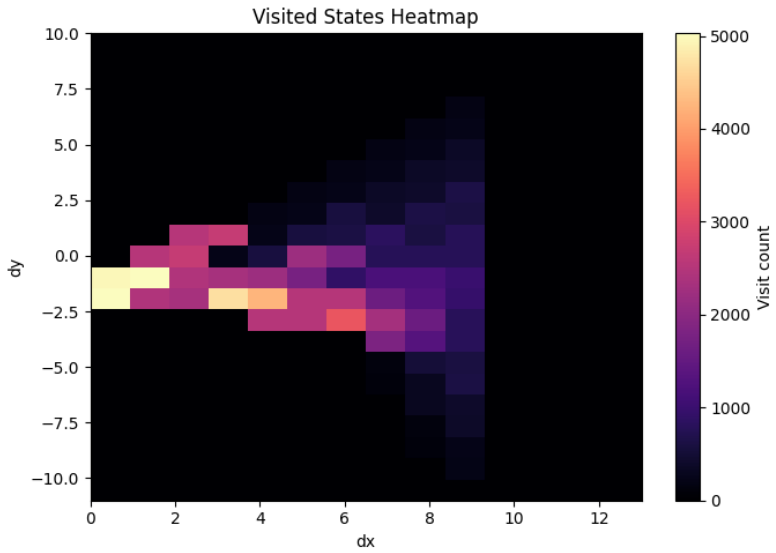


FIGURE 6 – Histogramme des états visités par l'agent SARSA(λ) en évaluation greedy (200 épisodes).

On constate bien que les états visités sont cohérents avec le jeu Flappy bird, les états du coins inférieur gauche ne sont pas visités malgré leur Q-value élevée.

3.6. Analyse

Avantages

— Mises à jour en ligne — signal propagé dès le premier épisode.
— Politique structurellement régulière grâce à l'approximation linéaire.
— Faible empreinte mémoire : 12 scalaires indépendamment de la taille de l'environnement.

Limitations

— **Instabilité** de l'entraînement : oscillations sans convergence durable.
— Biais lié à l'approximation de fonction.
— Sensibilité aux hyperparamètres α et λ.

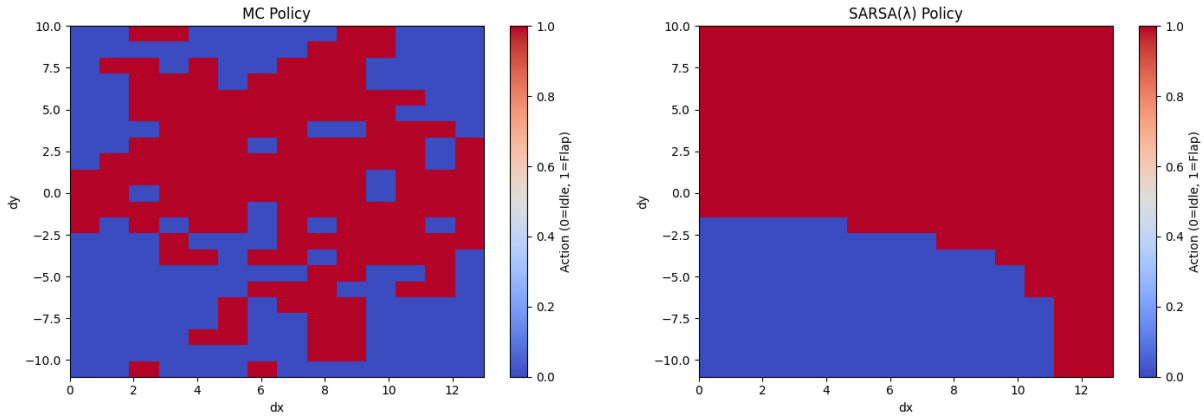
4. Comparaison des deux agents

4.1. Tableau récapitulatif

Table 4 – Comparaison synthétique des deux agents.

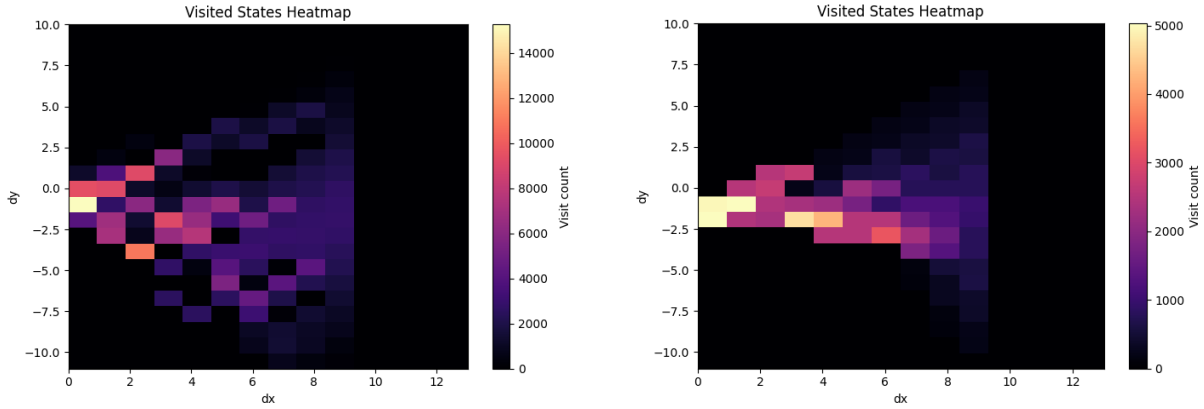
Critère	**MC Control**	**SARSA(λ)**
Famille	Monte Carlo	TD avec traces
Mise à jour	Fin d'épisode	En ligne (chaque pas)
Représentation Q	Tabulaire	Linéaire ($\mathbf{w}$)
Taille de Q	$O(\|S\| \times \|A\|)$	$O(d \times \|A\|) = 12$
Biais	Nul	Présent
Variance	Élevée	Plus faible (TD)
Politique	Fragmentée	Lisse, monotone
Stabilité	Très stable	Oscillante
Convergence	Garantie vers Q^*	Approximation de Q^*

4.2. Comparaison des politiques et des histogrammes



(a) Politique — MC Control. (b) Politique — SARSA(λ).

FIGURE 7 – Comparaison des politiques greedy apprises. La fragmentation de MC contraste avec la frontière lisse de SARSA(λ).



(a) MC Control. (b) SARSA(λ).

FIGURE 8 – histogramme des états visités en évaluation greedy (200 épisodes).

On constate que l'ensembles l'agent MC a une policy qui l'incite à visiter plus d'états que l'agent Sarsa. De plus l'histogramme de l'agent MC montre plus d'irrégularités ce qui s'explique par l'irrégularité de la policy apprise.

4.3. Compromis biais–variance

Compromis fondamental en RL

Monte Carlo est sans biais mais à haute variance : les retours G_t dépendent de trajectoires entières, qui fluctuent fortement.

SARSA(λ) réduit la variance grâce aux mises à jour TD, mais introduit un biais lié à l'approximation linéaire. Le paramètre λ contrôle l'interpolation : $\lambda = 0$ donne TD(0) pur et $\lambda = 1$ se rapproche de Monte Carlo.

4.4. Généralisation à de nouvelles configurations

La figure 9 présente les performances sur trois configurations non vues à l'entraînement.

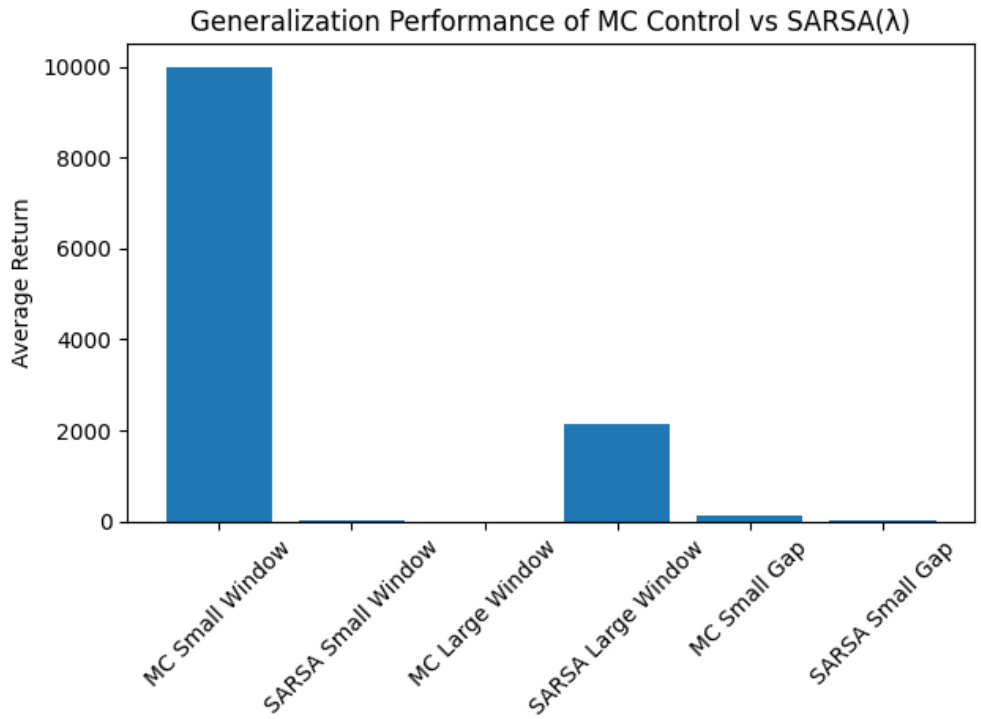


FIGURE 9 – Généralisation : retour moyen sur 100 épisodes greedy sur trois nouvelles configurations (*Small Window*, *Large Window*, *Small Gap*).

Les résultats sont frappants : MC Control surpasse largement SARSA(λ) sur la configuration avec une plus petite fenêtre (height=10, width=15, pipe gap=4) mais est moins bon pour la grande fenêtre (height=20, width=25,pipe gap= 4), atteignant 10 000 sur *Small Window* et 4.0 sur *Large Window*, tandis que SARSA(λ) atteint un average score de 2 000 sur *large window*.

Ces résultat, s'expliquent par :

— La politique tabulaire de MC a appris des décisions correctes pour tous les états visités, qui se retrouvent dans les nouvelles configurations. En revanche sur des états jamais visités le modèle n'a aucune idée de la policy à adopter.

— Le modèle linéaire de SARSA(λ) a une policy lisse et peut donc attribuer une policy cohérente à des états jamais visité qui sont proches d'états connus, ce qui explique la bonne performance sur *large window*

— Le modèle linéaire de SARSA(λ) est sensible au changement d'échelle des entrées : modifier la taille de la grille perturbe les features normalisés, ce qui expliqueratit la mauvaise performance de l'agent sarsa sur *small window*.

5. Structure du code

5.1. Interface commune

Table 5 – Description des fichiers du projet.

Fichier	**Rôle**
`text_flappy_bird_env_simple.py`	Définition de l'environnement Gymnasium
`train_mc_control_on_policy.py`	Agent 1 : entraînement, évaluation, visualisations
`train_sarsa_lambda_true_online.py`	Agent 2 : entraînement, évaluation, visualisations
`dev.ipynb`	Notebook de développement et d'expérimentation

```
# Entrainement
result = train(env=env, episodes=3000, gamma=0.99, ...)

# Evaluation greedy
avg_return = evaluate_greedy(env, Q_or_w, episodes=20)

# Evaluation sur configuration arbitraire
score = evaluate_on_config(Q_or_w, height=15, width=20, pipe_gap=4)

# Faire jouer l'agent (rendu textuel)
returns = run_greedy_agent(env, Q_or_w, episodes=1, render=True)

# Visualisations
plot_value_heatmap(env, Q_or_w)   # Heatmap V(s) = max_a Q(s,a)
plot_policy_heatmap(env, Q_or_w)  # Heatmap politique greedy
```

6. Conclusion et réponses à certaines questions

Ce projet a permis de mettre en oeuvre et de comparer deux familles fondamentales de l'apprentissage par renforcement sur un problème de contrôle concret.

Les résultats expérimentaux ont mis en évidence des différences marquées :

— **MC Control** converge de manière stable vers le score maximal et se montre logiquement robuste face à ine réduction de la window, en revanche lorsque la window grandit l'agent n'est plus capable de s'adapter. Une solution pour remédier à ce problème pourrait être de normaliser les observations (dx,dy).

— **SARSA(λ)** produit une politique plus régulière mais son entraînement est instable. Ses performances en généralisation sont bonnes sur une fenêtre plus grande grâce à sa Q-value éstimée linéairement. Cependant le modèle est moins bon sur une réstriction de fenêtre. Nous avons essayé différents paramètres d'apprentissage dans le but de stabiliser le learning, en vain. De plus ces tentatives rendaient la policy moins généralisables à des changements de taille de window.

Pourquoi avoir choisi l'environnement simple ?

L'environnement simple encode diretement les informations importantes de l'écran qui

sont utiles pour le déplacement de l'oiseau. L'environnement screen est plus lourd et ne représente à première vu pas d'avantage concret par rapport à l'environnement simple.

Nos agent sont ils adaptables à l'environnement réel RGB ?

Cet environnement est encore plus lourd que l'environnement screen, un modèle tabulaire comme l'agent 1 ne pourrait pas marcher sur un tel environnement car il a besoin de visiter de nombreuses fois tous les états. Il faudrait un nombre d'épisodes bien trop grand pour que notre agent puisse visiter touts les états. En revanche le modèle SARSA utilise une fonction d'évaluation linéaire avec une construction de features au préalable. Si la construction des features est bien adapté au nouvel environnement il serait alors envisageable d'utiliser cet agent